

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Corso di Elementi di Intelligenza Artificiale

MedExpert AI

Sistema Esperto di Diagnosi Medica

Implementazione in Prolog con Interfaccia Grafica Python

Docente:

Studenti:

Prof. Giancarlo Sperli

Luciano Meccariello N46007465

Gaspere Tortora N46007942

Andrea Francesco Bruno

N46007639

Anno Accademico 2025/2026

Indice

-
1. Obiettivo del Progetto
 2. Il Dominio: Diagnosi Medica
 3. Architettura del Sistema
 4. Knowledge Base in Prolog
 5. Regole di Inferenza
 6. Interfaccia Grafica (Python / Tkinter)
 7. Test e Validazione
 8. Conclusioni e Sviluppi Futuri
-

Documento generato automaticamente — Maggio 2026

1. Obiettivo del Progetto

L'obiettivo principale di questo progetto è la realizzazione di un **sistema esperto completo per la diagnosi medica**, che integri una base di conoscenza in Prolog con un'interfaccia grafica moderna in Python. Il sistema è progettato per dimostrare i concetti fondamentali dei sistemi esperti in un contesto applicativo realistico e didatticamente significativo.

1.1 Obiettivi Specifici

1. Progettare e implementare una **Knowledge Base** in Prolog contenente 17 patologie mediche con i rispettivi sintomi, categorie, descrizioni e trattamenti.
2. Implementare un **motore inferenziale** goal-driven (backward chaining) che calcoli, per ogni malattia, un **indice di copertura sintomatica** e ordini i risultati per valore decrescente.
3. Realizzare un meccanismo di **spiegazione** (*explanation facility*) che, per ogni diagnosi, espliciti quali sintomi della malattia sono stati trovati nel paziente e quali risultano mancanti.
4. Sviluppare un'**interfaccia grafica** intuitiva con tema medicale scuro, che permetta la selezione interattiva dei sintomi e la visualizzazione chiara dei risultati diagnostici.
5. Integrare Prolog e Python tramite **subprocess**, dimostrando l'interoperabilità tra paradigmi di programmazione (logico e imperativo).
6. Predisporre una **suite di test automatizzati** in Prolog per validare la correttezza delle diagnosi prodotte dal sistema.

1.2 Requisiti del Sistema

Il sistema deve soddisfare i seguenti requisiti funzionali e non funzionali:

Tipo	Requisito	Descrizione
Funzionale	Diagnosi multipla	Generare più diagnosi ordinate per certezza
Funzionale	Spiegazione	Fornire giustificazione per ogni diagnosi
Funzionale	Categorizzazione	Classificare le malattie per categoria medica
Non-funz.	Usabilità	Interfaccia intuitiva senza formazione necessaria
Non-funz.	Estensibilità	Aggiunta facile di nuove malattie alla KB
Non-funz.	Portabilità	Esecuzione su sistemi con Python 3 e SWI-Prolog

Tabella 1 — Requisiti funzionali e non funzionali del sistema.

2. Il Dominio: Diagnosi Medica

2.1 Perché la Diagnosi Medica?

La diagnosi medica rappresenta un dominio ideale per l'applicazione dei sistemi esperti per diverse ragioni fondamentali. In primo luogo, il processo diagnostico è intrinsecamente **basato su regole**: i medici utilizzano la propria esperienza e conoscenza per correlare insiemi di sintomi a possibili patologie, seguendo un ragionamento logico che può essere formalizzato in regole del tipo «se il paziente presenta i sintomi X, Y e Z, allora una diagnosi candidata è W».

In secondo luogo, la diagnosi medica è un problema di **copertura sintomatologica parziale**: raramente un paziente presenta tutti i sintomi caratteristici di una patologia, e spesso gli stessi sintomi possono essere associati a malattie diverse. Questa caratteristica si presta naturalmente a un meccanismo di ranking delle diagnosi, anziché a una risposta binaria.

Infine, la diagnosi medica richiede una **capacità di spiegazione**: non è sufficiente che il sistema produca una diagnosi, ma deve essere in grado di giustificarla, mostrando il ragionamento seguito. Questa trasparenza è fondamentale sia per la fiducia dell'utente che per scopi didattici.

2.2 Il Processo Diagnostico come Inferenza Logica

Il processo diagnostico può essere modellato come un problema di **inferenza logica**. Data una base di conoscenza che associa malattie a sintomi, e dato un insieme di sintomi osservati nel paziente, il sistema deve enumerare le diagnosi candidate e ordinarle. Formalmente, per ogni malattia M nella KB:

ICS(M) = round(|Sintomi_Paziente ∩ Sintomi_M| / |Sintomi_M| × 100)

L'**Indice di Copertura Sintomatica** (ICS) misura quale frazione dei sintomi tipici di M è presente nel quadro del paziente. È un indice di ranking, non una probabilità: le diagnosi a ICS più alto sono quelle che il sistema propone per prime, ma la decisione clinica finale resta del medico.

2.3 Patologie Coperte

MedExpert AI copre un ampio spettro di patologie comuni, organizzate in categorie mediche. Di seguito l'elenco completo delle **17 malattie** presenti nella Knowledge Base, organizzate in **10 categorie**:

#	Malattia	Categoria	N° Sintomi
1	Influenza	Respiratoria	7
2	COVID-19	Respiratoria	8
3	Bronchite	Respiratoria	7
4	Polmonite	Respiratoria	8
5	Gastrite	Gastrointestinale	6
6	Appendicite	Gastrointestinale	6
7	Reflusso gastroesofageo	Gastrointestinale	6
8	Emicrania	Neurologica	6
9	Meningite	Neurologica	8
10	Ipertensione	Cardiovascolare	6
11	Tachicardia	Cardiovascolare	6
12	Anemia	Ematologica	7
13	Diabete di tipo 2	Metabolica	6
14	Ipotiroidismo	Endocrina	7
15	Cistite	Urologica	6
16	Allergia stagionale	Immunitaria	6
17	Depressione	Psichiatrica	7

Tabella 2 — Elenco completo delle patologie nella Knowledge Base.

2.4 Inquadramento nel Corso

Il progetto si colloca nei **capitoli 7-10** del corso: rappresentazione della conoscenza in una base di conoscenza (KB), motore inferenziale, regole di produzione/inferenza e linguaggio Prolog basato su clausole di Horn. Il dominio medico è particolarmente naturale per questo paradigma: la conoscenza diagnostica può essere fornita da un esperto di dominio (il medico) sotto forma di regole tipo «se sono presenti i sintomi $s_1, \dots, s_n$ allora la malattia M è una diagnosi candidata», che si traducono direttamente in fatti e regole Prolog.

Il ruolo dell'esperto di dominio (in questo caso il medico) è cruciale: è lui a fornire la conoscenza diagnostica che alimenta la KB. Il motore inferenziale di SWI-Prolog si occupa poi di attivare le regole in modo *goal-driven* (backward chaining) per rispondere alle query dell'utente, mentre la GUI Python svolge il ruolo di interfaccia tra l'utente non-tecnico e il motore logico.

3. Architettura del Sistema

3.1 Panoramica Architeturale

MedExpert AI adotta un'architettura a **tre livelli** (three-tier), in cui ciascun livello ha responsabilità ben definite. Questa separazione garantisce modularità, manutenibilità e la possibilità di evolvere ciascun componente indipendentemente.

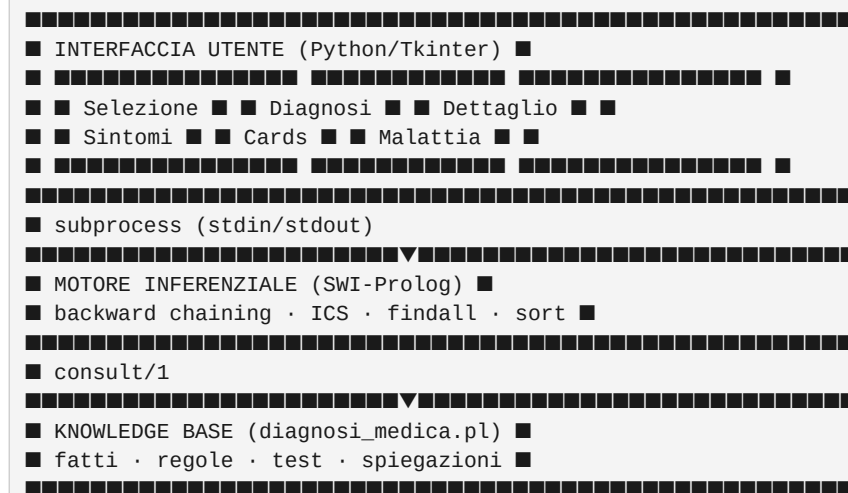


Figura 1 — Architettura a tre livelli di MedExpert AI.

3.2 Comunicazione tra Python e Prolog

L'integrazione tra Python e SWI-Prolog avviene tramite il modulo **subprocess** della libreria standard Python. Python avvia un processo SWI-Prolog, invia query formattate tramite standard input e analizza i risultati restituiti su standard output. Questo approccio è stato scelto per la sua semplicità, portabilità e assenza di dipendenze esterne (non richiede librerie di bridging come PySwip).

Il flusso di comunicazione è il seguente: (1) l'utente seleziona i sintomi nella GUI; (2) Python costruisce una query Prolog con la lista dei sintomi selezionati; (3) la query viene inviata a SWI-Prolog via subprocess; (4) Prolog esegue il backward chaining sulla KB e restituisce le diagnosi ordinate; (5) Python analizza l'output e aggiorna la GUI con i risultati.

3.3 Struttura dei File

File	Linguaggio	Ruolo
diagnosi_medica.pl	Prolog	Knowledge Base + Regole inferenziali + Test
interfaccia.py	Python	Interfaccia grafica Tkinter + ponte verso swipl
genera_pdf.py	Python	Generatore documentazione PDF
README.md	Markdown	Documentazione del progetto

Tabella 3 — Struttura dei file del progetto.

4. Knowledge Base in Prolog

4.1 Struttura dei Fatti

La Knowledge Base è organizzata utilizzando quattro predicati di fatti più un predicato derivato, ciascuno dei quali cattura un aspetto diverso della conoscenza medica:

Predicato	Arità	Descrizione	Esempio
sintomo/2	2	Associa una malattia a un singolo sintomo. Si dichiarano più fatti per la stessa malattia.	sintomo(influenza, febbre_alta).
categoria/2	2	Categoria medica della malattia.	categoria(influenza, respiratoria).
descrizione/2	2	Descrizione testuale della malattia.	descrizione(influenza, '...').
trattamento/2	2	Trattamento raccomandato.	trattamento(influenza, '...').
malattia/1	1	Predicato derivato: vero se la malattia esiste nella KB.	malattia(X) :- categoria(X, _).

Tabella 4 — Predicati della Knowledge Base.

4.2 Esempi di Fatti nella KB

Di seguito sono riportati alcuni esempi rappresentativi dei fatti codificati nella Knowledge Base Prolog:

```
%% Sintomi: un fatto per ogni coppia (malattia, sintomo).
sintomo(influenza, febbre_alta).
sintomo(influenza, mal_di_testa).
sintomo(influenza, dolori_muscolari).
sintomo(influenza, tosse_secca).
sintomo(influenza, mal_di_gola).
sintomo(influenza, stanchezza).
sintomo(influenza, naso_chiuso).

sintomo(covid19, febbre_alta).
sintomo(covid19, tosse_secca).
sintomo(covid19, difficolta_respiratorie).
sintomo(covid19, perdita_gusto_olfatto).
sintomo(covid19, stanchezza).
%% ...

%% Categorie.
categoria(influenza, respiratoria).
categoria(covid19, respiratoria).
categoria(diabete_tipo2, metabolica).
```

```
%% Predicato derivato.
malattia(X) :- categoria(X, _).
```

Listato 1 — Estratto della Knowledge Base in Prolog.

4.3 Scelte Progettuali della KB

La progettazione della Knowledge Base ha seguito alcuni principi fondamentali:

- **Fatti atomici anziché liste** — La scelta di codificare i sintomi come fatti separati (`sintomo(influenza, febbre_alta).`) riflette lo stile dichiarativo di Prolog: ogni fatto è un'unità minima di conoscenza, indipendentemente assertabile, e il motore di inferenza accede all'insieme dei sintomi di una malattia tramite `findall/3`. Questa rappresentazione facilita l'estensione della KB (aggiungere un nuovo sintomo a una malattia significa aggiungere una sola riga) e si presta naturalmente all'uso di predicati derivati come `malattia/1`.
- **Modularità** — Ogni aspetto della conoscenza (sintomi, descrizioni, trattamenti, categorie) è codificato in predicati separati, facilitando l'aggiornamento e l'estensione.
- **Uniformità** — Tutti i fatti seguono la stessa convenzione di naming: il primo argomento è sempre l'identificatore della malattia (atomo Prolog), il secondo contiene l'informazione associata.
- **Completezza** — Per ogni malattia sono forniti tutti e quattro i tipi di informazione (sintomi, descrizione, trattamento, categoria), garantendo che il sistema possa sempre fornire una risposta completa.

5. Regole di Inferenza

Il motore inferenziale di Prolog risolve i goal in modo **goal-driven**: a partire dal goal viene cercato un fatto o una regola la cui testa unifica con esso, e si tenta ricorsivamente di dimostrare le precondizioni nel body. Questo schema corrisponde al *backward chaining* della logica del primo ordine, paradigma per cui Prolog è stato pensato fin dalla sua origine.

In questo progetto la regola principale `diagnosi/3` sfrutta questo schema nel modo seguente: dato il goal `diagnosi(SintomiPaziente, Malattia, Certezza)`, l'interprete tenta di unificare `Malattia` con una malattia legittima della KB (tramite il predicato derivato `malattia/1`), raccoglie i suoi sintomi con `findall/3` e calcola l'ICS. La "catena" inferenziale qui non è multilivello — non ci sono regole di produzione che generano fatti intermedi inseriti nella KB — ma è la catena standard di unificazione e risoluzione interna a Prolog.

5.1 Indice di Copertura Sintomatica — `diagnosi/3`

La regola principale del sistema esperto è **`diagnosi/3`**, che sfrutta lo schema goal-driven di Prolog (backward chaining) per calcolare, per ogni malattia della KB, un indice di copertura sintomatica a partire dai sintomi forniti dall'utente.

```
%% diagnosi(+SintomiPaziente, ?Malattia, -Certezza)
%% Calcola l'indice di copertura sintomatica per una malattia.
diagnosi(SintomiPaziente, Malattia, Certezza) :-
    malattia(Malattia),
    sort(SintomiPaziente, SintomiUnici),
    findall(S, sintomo(Malattia, S), TuttiSintomi),
    intersection(SintomiUnici, TuttiSintomi, SintomiComuni),
    length(SintomiComuni, NComuni),
    NComuni > 0,
    length(TuttiSintomi, NTotali),
    Certezza is round((NComuni / NTotali) * 100).
```

Listato 2 — Regola `diagnosi/3` estratta da `diagnosi_medica.pl`.

Alcuni dettagli implementativi che è utile commentare:

- *sort/2* rimuove eventuali sintomi duplicati dall'input dell'utente prima del confronto, rendendo la regola idempotente rispetto alle ripetizioni.
- *findall/3* raccoglie *tutti* i sintomi associati alla malattia interrogando ripetutamente *sintomo/2*, restituendo una lista. È il punto in cui si materializza l'insieme dei sintomi a partire dai fatti atomici della KB.
- Il risultato è **arrotondato all'intero** (*round*) anziché lasciato in decimali, per coerenza con i valori mostrati nella GUI.
- La clausola *NComuni > 0* esclude le malattie senza alcun sintomo in comune con il paziente, evitando di mostrare diagnosi totalmente irrilevanti.

5.2 Indice di Copertura Sintomatica (ICS)

Per ogni malattia *M* nella Knowledge Base, il sistema calcola un **Indice di Copertura Sintomatica** definito come:

$$ICS(M) = \text{round} \left(\frac{|S_{\text{paziente}} \cap S_M|}{|S_M|} \times 100 \right)$$

dove *S_paziente* è l'insieme dei sintomi selezionati dall'utente, *S_M* è l'insieme dei sintomi associati alla malattia *M* nella KB, e l'arrotondamento è all'intero più vicino.

L'ICS misura **in che proporzione i sintomi tipici di *M* sono presenti nel paziente**. In termini statistici è una grandezza affine al *recall* della malattia *M* rispetto ai sintomi: non penalizza i sintomi del paziente che non sono associati a *M*. Per questo motivo l'ICS va inteso come **strumento di ranking diagnostico**, non come probabilità a posteriori di *M* dato il quadro sintomatologico. Il ranking è prodotto da *diagnosi_ordinate/2*, che aggrega le diagnosi e le ordina per ICS decrescente.

Esempio. Se l'influenza ha 7 sintomi nella KB e il paziente ne presenta 3 corrispondenti, $ICS(\text{influenza}) = \text{round}(3/7 \times 100) = 43\%$.

5.3 Diagnosi Ordinate — *diagnosi_ordinate/2*

La regola ***diagnosi_ordinate/2*** raccoglie tutte le diagnosi possibili e le ordina per ICS decrescente, presentando le diagnosi a copertura più alta per prime:

```
%% diagnosi_ordinate(+Sintomi, -DiagnosiOrdinate)
%% Raccoglie le diagnosi e le ordina per ICS decrescente.
diagnosi_ordinate(Sintomi, DiagnosiOrdinate) :-
    findall(
        certezza(CF, Malattia),
        diagnosi(Sintomi, Malattia, CF),
        Diagnosi
    ),
    sort(0, @>=, Diagnosi, DiagnosiOrdinate).
```

Listato 3 — Ordinamento delle diagnosi (il termine certezza/2 e la variabile CF sono identificatori interni al codice Prolog).

5.4 Facility di Spiegazione — *spiega_diagnosi/4*

Una caratteristica fondamentale dei sistemi esperti è la capacità di **spiegare** il ragionamento che ha portato a una conclusione. La regola ***spiega_diagnosi/4*** divide i sintomi della malattia in due liste: quelli effettivamente *trovati* nel paziente e quelli *mancanti*.

```
%% spiega_diagnosi(+SintomiPaziente, +Malattia, -Trovati, -Mancanti)
%% Divide i sintomi della malattia in trovati nel paziente e mancanti.
spiega_diagnosi(SintomiPaziente, Malattia, Trovati, Mancanti) :-
    sort(SintomiPaziente, SintomiUnici),
    findall(S, sintomo(Malattia, S), Tutti),
```



```
intersection(SintomiUnici, Tutti, Trovati),
subtract(Tutti, Trovati, Mancanti).
```

Listato 4 — Regola spiega_diagnosi/4 estratta da diagnosi_medica.pl.

La facility di spiegazione restituisce due liste: i **sintomi trovati**, cioè quelli del paziente che sono effettivamente associati alla malattia M, e i **sintomi mancanti**, cioè quelli associati a M che il paziente non ha riportato. Questa rappresentazione è quella consumata direttamente dalla GUI per il pannello "Dettagli", che mostra esplicitamente sia il match sia ciò che resta inspiegato — un elemento di trasparenza diagnostica.

5.5 Esempio Passo-Passo di Inferenza

Si consideri un paziente che presenta i seguenti tre sintomi: **febbre_alta**, **tosse_secca**, **stanchezza**. Il sistema esegue la regola *diagnosi/3* per ciascuna malattia della KB e produce l'output ordinato che segue (valori ottenuti eseguendo davvero la query *diagnosi_ordinate/2* sul codice).

Malattia	N° sintomi totali	Sintomi corrispondenti	ICS
Influenza	7	3 (febbre_alta, tosse_secca, stanchezza)	43%
COVID-19	8	3 (febbre_alta, tosse_secca, stanchezza)	38%
Polmonite	8	2 (febbre_alta, stanchezza)	25%
Meningite	8	2 (febbre_alta, stanchezza)	25%
Tachicardia	6	1 (stanchezza)	17%
Reflusso gastroesofageo	6	1 (tosse_secca)	17%
Bronchite	7	1 (stanchezza)	14%
...	...	...	...

Tabella 5 — Output reale di *diagnosi_ordinate/2* per il quadro sintomatologico {febbre_alta, tosse_secca, stanchezza}. Sono mostrate le 7 diagnosi a copertura più alta; il sistema restituisce in totale 14 candidate.

Il risultato finale presentato all'utente è **Influenza (43%)** come diagnosi più probabile, seguita da COVID-19 e dalle altre diagnosi a copertura inferiore. Il sistema fornisce inoltre, per ogni diagnosi cliccata, l'elenco dei sintomi *trovati* e *mancanti* tramite la facility di spiegazione (*spiega_diagnosi/4*).

6. Interfaccia Grafica (Python / Tkinter)

6.1 Filosofia di Design

L'interfaccia grafica di MedExpert AI è stata progettata seguendo una filosofia di **dark medical theme**, ispirata alle interfacce professionali dei software medici. La scelta di un tema scuro è motivata da considerazioni di usabilità: riduce l'affaticamento visivo durante sessioni prolungate e conferisce un aspetto professionale al sistema.

La palette cromatica è basata su tonalità di blu scuro e accenti ciano, con testi chiari su sfondo scuro. I colori principali utilizzati sono:

Elemento	Colore	Codice Hex	Utilizzo
Sfondo principale	Blu molto scuro	#0a0e17	Background della finestra

Elemento	Colore	Codice Hex	Utilizzo
Pannelli	Blu scuro	#111827	Sfondo dei pannelli laterali
Accento primario	Ciano	#00d4ff	Bordi, titoli, elementi attivi
Accento secondario	Ciano chiaro	#00e5ff	Hover, selezioni
Testo primario	Bianco	#ffffff	Titoli e testi principali
Testo secondario	Grigio chiaro	#94a3b8	Testi descrittivi
Successo	Verde	#00ff88	Certezza alta, conferme
Attenzione	Giallo	#ffaa00	Certezza media, avvisi

Tabella 6 — Palette cromatica dell'interfaccia.

6.2 Flusso di Interazione

Il flusso di interazione dell'utente con il sistema segue un percorso lineare e intuitivo:

1. L'utente avvia l'applicazione eseguendo `python3 interfaccia.py`.
2. Nel pannello sinistro, seleziona i sintomi presenti spuntando le checkbox corrispondenti.
3. Clicca il pulsante «Esegui Diagnosi» per avviare l'analisi.
4. Il sistema invia la query a Prolog via subprocess e riceve i risultati.
5. Le diagnosi appaiono nel pannello centrale come card cliccabili.
6. Cliccando su una card, il pannello destro mostra i dettagli completi.
7. L'utente può premere «Reset» per cancellare tutto e ricominciare.

6.3 Architettura del Codice Python

Il codice in `interfaccia.py` è organizzato attorno a due classi: **MedExpertApp**, che costruisce la finestra Tkinter e ne gestisce eventi e stato, e **PrologEngine**, che fa da ponte verso `swipl`. I metodi principali di **MedExpertApp** sono:

Metodo	Descrizione
<code>__init__()</code>	Inizializza finestra, font, palette, stato dell'app e istanzia PrologEngine.
<code>_build_header() / _build_body() / _build_footer()</code>	Costruzione dei tre blocchi principali della UI.
<code>_build_left_panel()</code>	Pannello sinistro con le checkbox dei sintomi raggruppate per categoria.
<code>_build_center_panel()</code>	Area centrale che ospita le card delle diagnosi.
<code>_build_right_panel()</code>	Pannello destro dei dettagli della malattia selezionata.
<code>_on_analyze()</code>	Lancia la diagnosi in un thread separato per non bloccare la UI.
<code>_on_analysis_done(results)</code>	Callback che aggiorna la UI con i risultati.
<code>_on_card_click(malattia)</code>	Click su una card: richiede a Prolog la spiegazione e popola il pannello destro.
<code>_on_reset()</code>	Resetta checkbox e pannelli.
<code>_toggle_fullscreen() / _exit_fullscreen()</code>	Gestione fullscreen (F11 / Esc).

Tabella 7 — Metodi principali di MedExpertApp.

Classe	Descrizione
PrologEngine	Ponte verso swipl: avvia il processo via subprocess, esegue query_diagnosi e query_spiega sul file diagnosi_medica.pl, fa il parsing dell'output testuale.

Tabella 7b — Classe ausiliaria di interfaccia.py.

7. Test e Validazione

7.1 Suite di Test

MedExpert AI include una suite di test automatizzati scritta direttamente in Prolog, integrata nel file della Knowledge Base. I test verificano la correttezza del motore inferenziale per diverse combinazioni di sintomi e per i casi limite (input duplicato, sintomi inesistenti, ordinamento dei risultati).

Per eseguire la suite di test si usa il seguente comando:

```
swipl -g "test_diagnosi, halt" diagnosi_medica.pl
```

7.2 Casi di Test

Di seguito i 7 test reali presenti nella KB con lo scopo di ciascuno e l'esito atteso. Tutti i valori numerici sono stati verificati eseguendo la suite sul codice reale.

#	Nome	Scopo	Esito atteso
1	Corrispondenza esatta	Tutti e 7 i sintomi dell'influenza in input	ICS(influenza) = 100%
2	Corrispondenza parziale	3 sintomi su 7 dell'influenza	ICS(influenza) = round(3/7·100) = 43%
3	Diagnosi multiple	Sintomi condivisi: febbre_alta, mal_di_testa, stanchezza	≥ 2 diagnosi restituite
4	Nessuna corrispondenza	Sintomi inventati / non presenti nella KB	0 diagnosi
5	Gestione duplicati	Stesso sintomo ripetuto nell'input	Stessa ICS della versione senza duplicati
6	Spiegazione diagnosi	spiega_diagnosi/4 con 2 sintomi dell'influenza	2 trovati, 5 mancanti
7	Ordinamento risultati	diagnosi_ordinate/2 con 4 sintomi misti	Lista ordinata per ICS decrescente

Tabella 8 — Suite di test (test_1 ... test_7).

7.3 Codice di Test in Prolog

```
%% Test runner principale.
test_diagnosi :-
    format(' SUITE DI TEST - SISTEMA DIAGNOSTICO~n'),
    test_1(R1), test_2(R2), test_3(R3),
    test_4(R4), test_5(R5), test_6(R6), test_7(R7),
    somma_risultati([R1,R2,R3,R4,R5,R6,R7], Sup, Tot),
    Fail is Tot - Sup,
    format(' RIEPILOGO: ~w/~w superati, ~w falliti~n',
    [Sup, Tot, Fail]).

%% Esempio: test_2 verifica la corrispondenza parziale.
%% Influenza ha 7 sintomi, ne passiamo 3 -> ICS attesa = 43%.
test_2(Risultato) :-
    SintomiPaziente = [febbre_alta, tosse_secca, stanchezza],
    diagnosi(SintomiPaziente, influenza, Certezza),
    CertezzaAttesa is round((3 / 7) * 100),
    ( Certezza == CertezzaAttesa
```

```
-> Risultato = 1
; Risultato = 0
).
```

Listato 5 — Estratto della suite di test in Prolog.

7.4 Risultati della Validazione

Tutti i 7 test case sono stati eseguiti con successo (output reale: RIEPILOGO: 7/7 superati, 0 falliti). Il sistema produce diagnosi con valori di ICS coerenti con la sovrapposizione tra sintomi del paziente e sintomi delle malattie. La suite serve anche come **regression testing**: a ogni modifica della KB i test verificano che le diagnosi precedentemente corrette non siano state compromesse. Il predicato `somma_risultati/3` produce un riepilogo finale *superati / totale*.

8. Conclusioni e Sviluppi Futuri

8.1 Riepilogo dei Risultati

Il progetto MedExpert AI ha dimostrato con successo la fattibilità e l'efficacia di un sistema esperto per la diagnosi medica, implementato combinando la potenza della programmazione logica in Prolog con l'accessibilità di un'interfaccia grafica Python. I principali risultati ottenuti sono:

- **Knowledge Base completa** — È stata sviluppata una KB contenente 17 patologie distribuite su 10 categorie mediche, con 44 sintomi distinti, descrizioni dettagliate e trattamenti raccomandati.
- **Motore inferenziale funzionale** — Lo schema goal-driven di Prolog (backward chaining), combinato con il calcolo dell'**Indice di Copertura Sintomatica**, produce una lista di diagnosi candidate ordinate, come confermato dalla suite di test.
- **Explanation Facility** — Il sistema è in grado di spiegare ogni diagnosi, mostrando esplicitamente i sintomi *trovati* e i sintomi *manca*nti della malattia, soddisfacendo un requisito fondamentale dei sistemi esperti.
- **Interfaccia utente professionale** — La GUI con tema medico scuro offre un'esperienza utente intuitiva e visivamente accattivante, rendendo il sistema accessibile anche a utenti non tecnici.
- **Integrazione multi-paradigma** — L'integrazione tra Prolog (paradigma logico) e Python (paradigma imperativo/OOP) dimostra la complementarità dei diversi approcci alla programmazione nell'ambito dell'AI.

8.2 Sviluppi Futuri

Il sistema può essere esteso e migliorato in diverse direzioni:

Area	Sviluppo Proposto	Impatto
Knowledge Base	Espansione a 50+ malattie con sintomi più granulari	Maggiore copertura diagnostica
Inferenza	Introduzione di reti bayesiane per gestione incertezza avanzata	Diagnosi più accurate con correlazioni tra sintomi
Inferenza	Forward chaining per diagnosi proattiva basata su profilo paziente	Rilevamento precoce di patologie
GUI	Interfaccia web con Flask/Django per accesso remoto	Accessibilità da qualsiasi dispositivo

Area	Sviluppo Proposto	Impatto
GUI	Visualizzazione grafica delle relazioni sintomo-malattia	Migliore comprensione del ragionamento
Dati	Integrazione con database medici (ICD-10, SNOMED CT)	Standard clinici e interoperabilità
AI ibrida	Combinazione con ML per apprendimento da casi clinici	Miglioramento continuo della KB
NLP	Input in linguaggio naturale per descrizione sintomi	Interazione più naturale con l'utente

Tabella 9 — Possibili sviluppi futuri del sistema.

8.3 Connessione ai Temi del Corso

In conclusione, MedExpert AI rappresenta un'applicazione concreta dei principali temi affrontati nel corso: **rappresentazione della conoscenza** in una KB come insieme di sentence, **inferenza nella logica del primo ordine** con *forward* e *backward chaining*, e **Prolog** come linguaggio basato su clausole di Horn con motore inferenziale goal-driven. Il progetto realizza l'architettura classica di un sistema esperto: KB + esperto di dominio (gli autori in veste di *knowledge engineer* medico) + motore inferenziale (SWI-Prolog).

Il sistema dimostra che l'AI simbolica, nonostante l'attuale predominanza degli approcci basati su apprendimento automatico, mantiene un ruolo fondamentale in applicazioni dove la spiegabilità, la trasparenza e il determinismo sono requisiti imprescindibili — come nel caso della diagnosi medica. La combinazione di Prolog per il ragionamento logico e Python per l'interfaccia utente rappresenta un esempio efficace di come diversi paradigmi di programmazione possano essere integrati per realizzare sistemi AI completi e funzionali.

«L'intelligenza artificiale non è un sostituto dell'intelligenza umana, ma uno strumento per amplificarla.»

Luciano Meccariello N46007465
Gaspere Tortora N46007942
Andrea Francesco Bruno N46007639